

Assignment 14

Post-Quantum Cryptography Implementation and Testing

Methodology

This project aimed to implement and evaluate post-quantum cryptographic primitives to assess their readiness for real-world security systems. The notebook included:

1. Kyber: Used for post-quantum key encapsulation and decapsulation.
2. Dilithium: Employed for digital signature generation and verification.
3. AES: Used for symmetric encryption/decryption of messages after key exchange.
4. Tampering Tests: Simulated man-in-the-middle attacks and message alterations to evaluate integrity.
5. All cryptographic components were implemented in modular functions to support clarity and reusability.

Code Overview

Kyber Module: Generated public-private key pairs, encapsulated a shared secret, and performed decapsulation for key agreement.

```
# Step 1: Clone and build pyoqs (this takes ~5 minutes)
```

```
!apt-get install -y cmake gcc libssl-dev
```

```
!git clone --recursive
```

```
https://github.com/open-quantum-safe/liboqs-python.git
```

```
%cd liboqs-python
```

```
!python3 setup.py build
```

```
!python3 setup.py install
```

```
# Step 2: Now use pyoqs for Kyber512 key generation & encapsulation
```

```
import oqs
```

```
# Generate a Kyber512 keypair
```

```
with oqs.KeyEncapsulation('Kyber512') as kem:
```

```
    public_key = kem.generate_keypair()
```

```
# Simulate sender encapsulating a shared secret using public key
```

```
ciphertext, shared_secret_sender = kem.encap_secret(public_key)
```

```
# Recipient decapsulates to retrieve the same shared secret
```

```
shared_secret_recipient = kem.decap_secret(ciphertext)
```

```
print("Sender's Shared Secret: ", shared_secret_sender.hex())
```

```

print("Recipient's Shared Secret:", shared_secret_recipient.hex())

if shared_secret_sender == shared_secret_recipient:
    print("Success! Shared secrets match.")
else:
    print("Failure! Shared secrets do not match.")

```

Dilithium Module: Signed messages using the private key and verified them using the public key.

Step 2: Use Dilithium from pyoqs

```
import oqs
```

The message to sign

```
message = b"Post-quantum secure!"
```

Generate a Dilithium key pair

```

with oqs.Signature('Dilithium2') as signer:
    public_key = signer.generate_keypair()
    private_key = signer.export_secret_key()

```

Sign the message

```

signature = signer.sign(message)
print("Message signed successfully!")

```

Verify the signature with the public key

```

verified = signer.verify(message, signature, public_key)
print("Signature valid on original message:", verified)

```

Modify the message

```
tampered_message = b"Post-quantum is cool!"
```

Verify the signature on the tampered message (should fail)

```

verified_tampered = signer.verify(tampered_message, signature,
public_key)
print("Signature valid on tampered message:", verified_tampered)

```

```

Message signed successfully!
Signature valid on original message: True
Signature valid on tampered message: False

```

AES Module: Encrypted a plaintext using the agreed key and decrypted it, verifying message integrity.

Step 2: Import libraries

```
import oqs
import os
import hashlib
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
```

Step 3: Generate Kyber512 keypair

```
with oqs.KeyEncapsulation('Kyber512') as kem:
```

Recipient generates Kyber keypair

```
public_key = kem.generate_keypair()
```

Sender encapsulates a shared secret

```
ciphertext, shared_secret_sender = kem.encap_secret(public_key)
```

Derive AES key (256-bit) from shared secret using SHA-256

```
aes_key_sender = hashlib.sha256(shared_secret_sender).digest()
```

Encrypt message using AES-GCM

```
aesgcm = AESGCM(aes_key_sender)
```

```
nonce = os.urandom(12) # AES-GCM standard nonce size
```

```
plaintext = b"Confidential post-quantum message."
```

```
ciphertext_encrypted = aesgcm.encrypt(nonce, plaintext, None)
```

```
print("Sender encrypted message.")
```

Receiver decapsulates shared secret from ciphertext

```
shared_secret_receiver = kem.decap_secret(ciphertext)
```

```
aes_key_receiver = hashlib.sha256(shared_secret_receiver).digest()
```

Decrypt message using AES-GCM

```
aesgcm_receiver = AESGCM(aes_key_receiver)
```

```
try:
```

```
    decrypted = aesgcm_receiver.decrypt(nonce, ciphertext_encrypted,
None)
```

```
    print("Decrypted message:", decrypted.decode())
```

```
    print("Integrity verified (authentication tag is valid).")
```

```
except Exception as e:
```

```
    print("Decryption failed or integrity check failed:", str(e))
```

```
Sender encrypted message.  
Decrypted message: Confidential post-quantum message.  
Integrity verified (authentication tag is valid).
```

Tampering Tests: Modified ciphertexts and signatures to observe failure points in decryption and verification.

Step 2: Python code for tampering test

```
import oqs  
import os  
import hashlib  
from cryptography.hazmat.primitives.ciphers.aead import AESGCM  
  
print("\nStarting key encapsulation and encryption process...")  
  
with oqs.KeyEncapsulation('Kyber512') as kem:  
    # Generate Kyber key pair  
    public_key = kem.generate_keypair()  
  
    # Sender encapsulates secret  
    kyber_ciphertext, shared_secret_sender = kem.encap_secret(public_key)  
    aes_key_sender = hashlib.sha256(shared_secret_sender).digest()  
  
    # AES-GCM encrypt  
  
    aesgcm = AESGCM(aes_key_sender)  
    nonce = os.urandom(12)  
    message = b"Secure message with integrity."  
    ciphertext = aesgcm.encrypt(nonce, message, None)  
  
    print("Message encrypted with AES-GCM.")  
  
    # -----  
    # ❶ Test tampering with AES ciphertext  
    # -----  
    tampered_ciphertext = bytearray(ciphertext)  
    tampered_ciphertext[5] ^= 0xFF # Flip a bit  
    tampered_ciphertext = bytes(tampered_ciphertext)  
  
    print("\n Testing tampered AES-GCM ciphertext...")
```

```

try:
    # Decapsulate normally
    shared_secret_receiver = kem.decap_secret(kyber_ciphertext)
    aes_key_receiver = hashlib.sha256(shared_secret_receiver).digest()

    # Attempt decryption
    aesgcm_receiver = AESGCM(aes_key_receiver)
    decrypted = aesgcm_receiver.decrypt(nonce, tampered_ciphertext,
None)
    print("Unexpected success: Decrypted message:", decrypted.decode())
except Exception as e:
    print("AES-GCM detected tampering. Decryption failed:", str(e))

# -----
# Test tampering with Kyber ciphertext
# -----
print("\nTesting tampered Kyber ciphertext...")

tampered_kem_ct = bytearray(kyber_ciphertext)
tampered_kem_ct[0] ^= 0xAA # Flip a bit
tampered_kem_ct = bytes(tampered_kem_ct)

try:
    # Attempt decapsulation with tampered ciphertext
    shared_secret_tampered = kem.decap_secret(tampered_kem_ct)
    aes_key_fake = hashlib.sha256(shared_secret_tampered).digest()

    # Try decrypting original ciphertext with incorrect key
    aesgcm_fake = AESGCM(aes_key_fake)
    decrypted = aesgcm_fake.decrypt(nonce, ciphertext, None)
    print("Unexpected success: Decrypted message:", decrypted.decode())
except Exception as e:
    print("Kyber tampering detected. Decryption or tag check failed:",
str(e))

```

```
Starting key encapsulation and encryption process...  
Message encrypted with AES-GCM.
```

```
Testing tampered AES-GCM ciphertext...  
AES-GCM detected tampering. Decryption failed:
```

```
Testing tampered Kyber ciphertext...  
Kyber tampering detected. Decryption or tag check failed:
```

Significance of Post-Quantum Security

As quantum computing advances, traditional algorithms like RSA and ECC become vulnerable. Kyber and Dilithium, being part of the NIST post-quantum cryptography standardization, offer strong resistance against quantum attacks.